

# Looping dan Rekursif

Arsitektur Repetisi Komputasi  
dan Logika Aliran Data di R

Disusun oleh:

Weksi Budiaji & Ferdian Bangkit Wijaya

# Filosofi Repetisi: Otak Manusia vs. Arsitektur CPU

## Manusia (Kognitif)



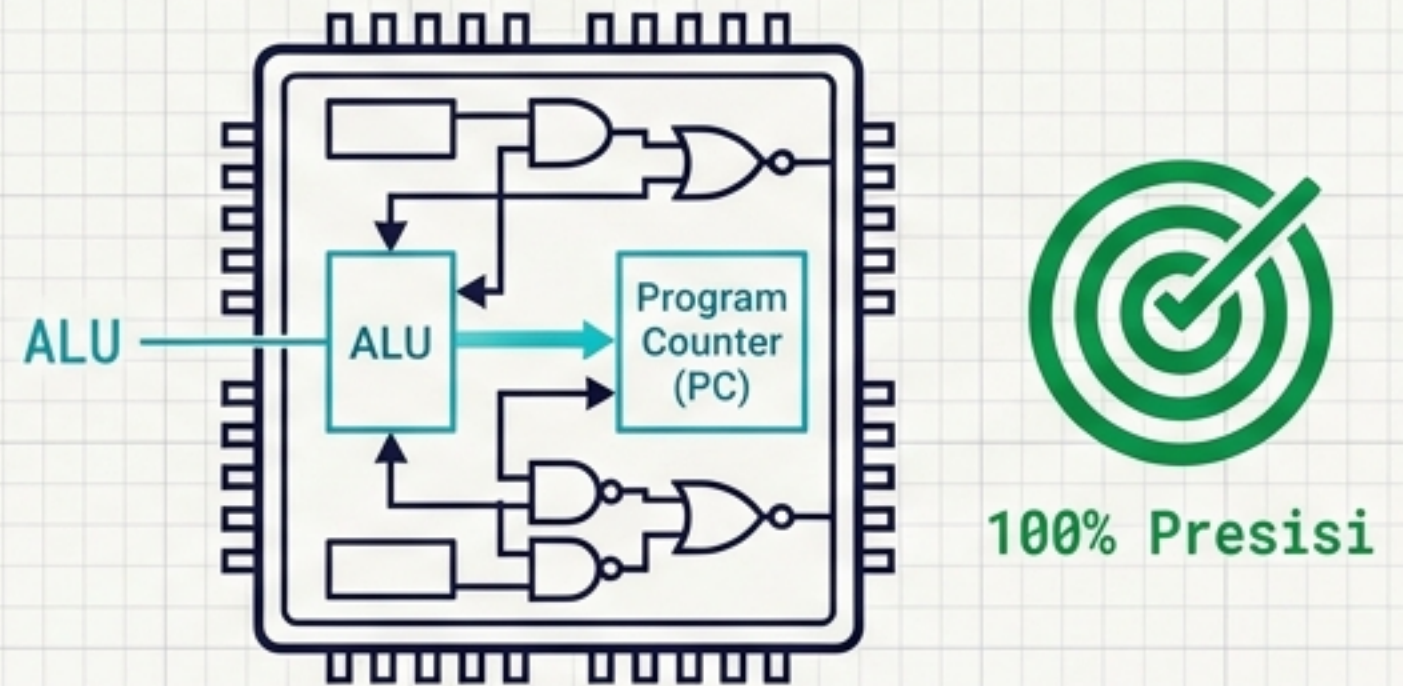
Rentan terhadap cognitive fatigue dan degradasi akurasi.

**Tugas:** Menghitung `running mean 100.000` data.

**Waktu Estimasi:** `~277 Jam`

**Presi:** `Menurun drastis` seiring waktu.

## Komputer (Hardware)



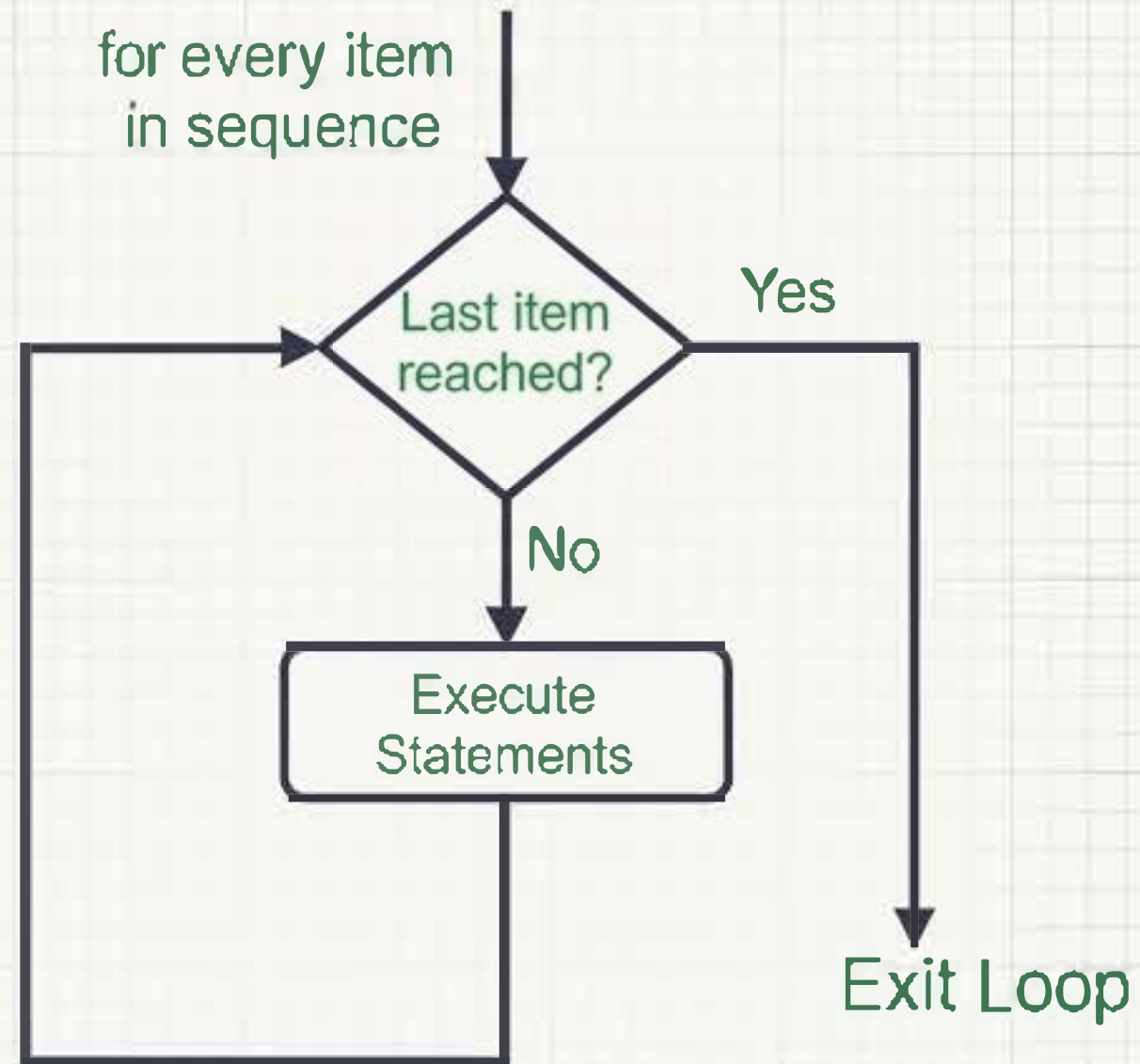
Sirkuit digerakkan oleh `Clock Speed (GHz)`.  
`Bebas lelah`.

**Tugas:** `Kalkulasi 100.000` data di `R`.

**Waktu Eksekusi:** `0.004 Detik`

**Presi:** `Konsisten 100%` tanpa degradasi.

# Looping & Struktur For Loop



## For Loop in R

*For loop in R*

## Anatomi Repetisi

**Inisialisasi** Counter awal didaftarkan di RAM

**Kondisi Batas** Evaluasi terminasi oleh ALU

**Badan Loop** Eksekusi instruksi utama

**Update Nilai** Langkah inkrementasi/dekrementasi



### BAHAYA ALOKASI DINAMIS DI R

Jangan gunakan penggabungan: `v <- c(v, x)`

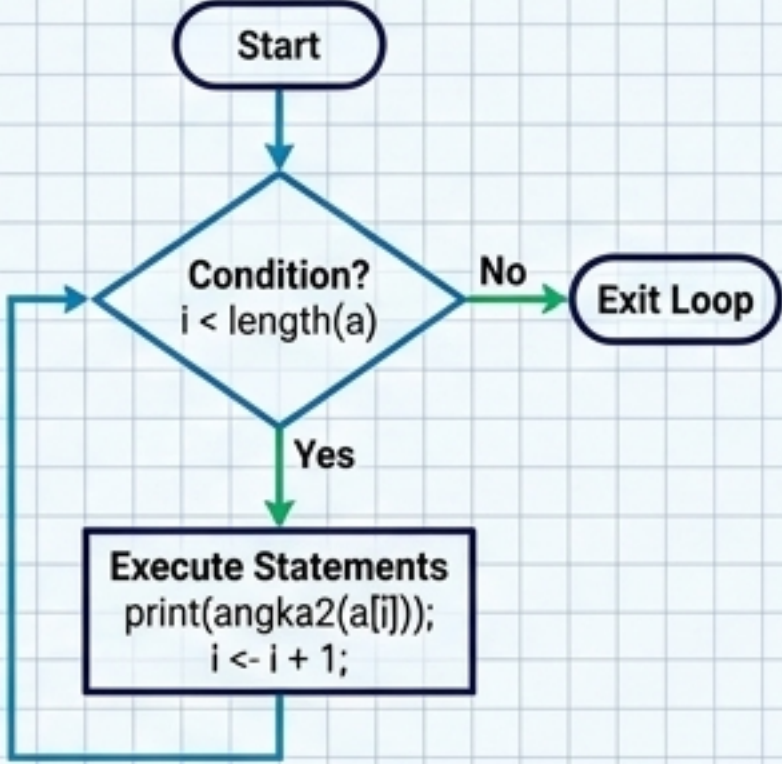
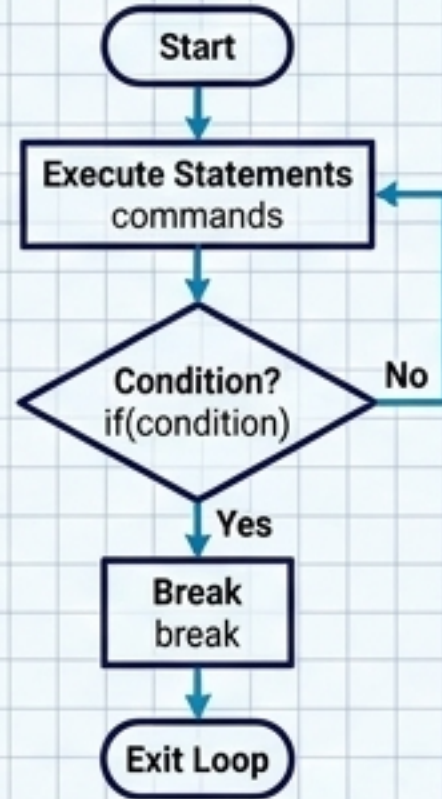
**Kompleksitas  $O(N^2)$**  - Memaksa RAM menyalin ulang seluruh data terus-menerus!

**Solusi Emas:** Lakukan Pre-allocation (contoh: `vector('numeric', N)`). Kompleksitas menjadi linier  $O(N)$ .

**Bug Logika:** Hindari `1:length(x)` untuk data kosong.

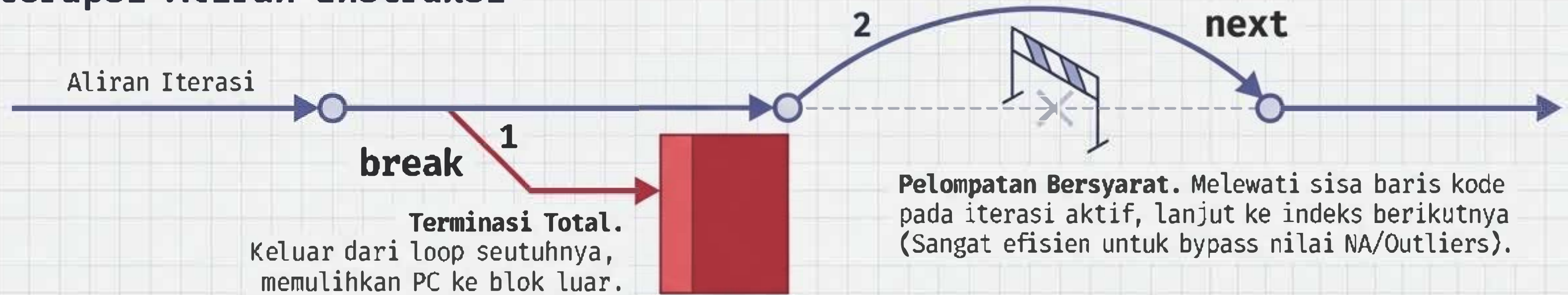
Selalu gunakan `seq_along(x)`.

# Diagnostic Matrix: Evaluasi while vs repeat

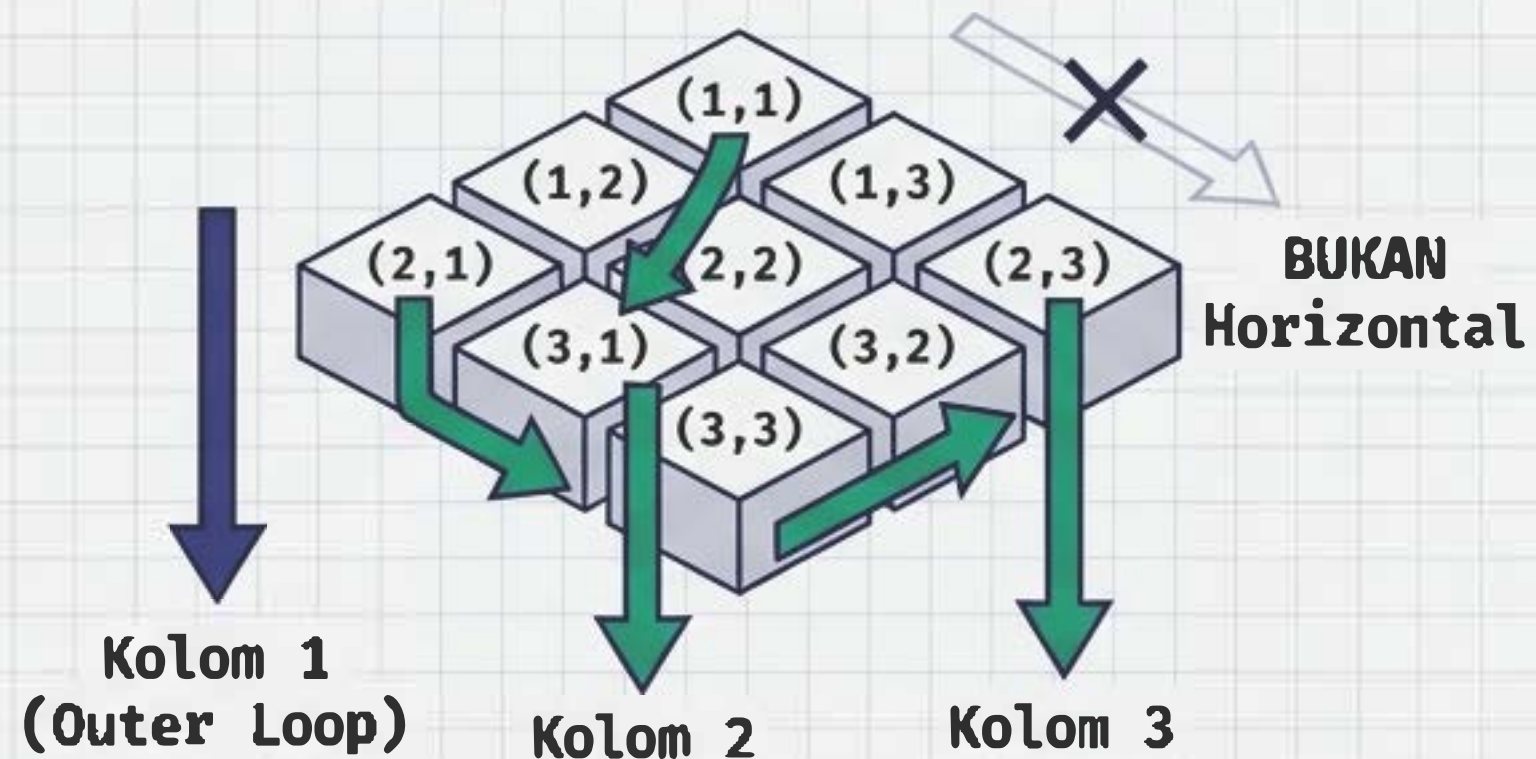
| While Loop                                                                                                                                                                                                                                                                                                            | Repeat Loop                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <pre>graph TD; Start([Start]) --&gt; Cond{Condition?<br/>i &lt; length(a)}; Cond -- No --&gt; Exit([Exit Loop]); Cond -- Yes --&gt; Exec[Execute Statements<br/>print(angka2(a[i]));<br/>i &lt;- i + 1;]; Exec --&gt; Cond;</pre> |  <pre>graph TD; Start([Start]) --&gt; Exec[Execute Statements<br/>commands]; Exec --&gt; Cond{Condition?<br/>if(condition)}; Cond -- Yes --&gt; Break[Break<br/>break]; Break --&gt; Exit([Exit Loop]); Cond -- No --&gt; Exec;</pre>                               |
| <p><b>Mekanika:</b> Pre-test loop (Dievaluasi sebelum masuk blok).</p> <p><b>Eksekusi Minimal:</b> Bisa 0 kali (jika kondisi langsung FALSE).</p> <p><b>Risiko Sistem:</b> Fatal error jika evaluasi kondisi bernilai NA.</p> <p><b>Kasus Ideal:</b> Iterasi konvergensi dengan kondisi batas yang aman.</p>          | <p><b>Mekanika:</b> Post-test loop (Dievaluasi di dalam blok via 'break').</p> <p><b>Eksekusi Minimal:</b> Dijamin minimal 1 kali.</p> <p><b>Risiko Sistem:</b> Risiko Infinite Loop sangat tinggi jika kondisi break tidak tercapai.</p> <p><b>Kasus Ideal:</b> Pencarian akar Newton-Raphson (wajib mengeksekusi rumus sebelum evaluasi selisih).</p> |

# Kendali Aliran (break/next) & Optimalisasi Nested Loop

## Interupsi Aliran Instruksi



## Arsitektur Memori R



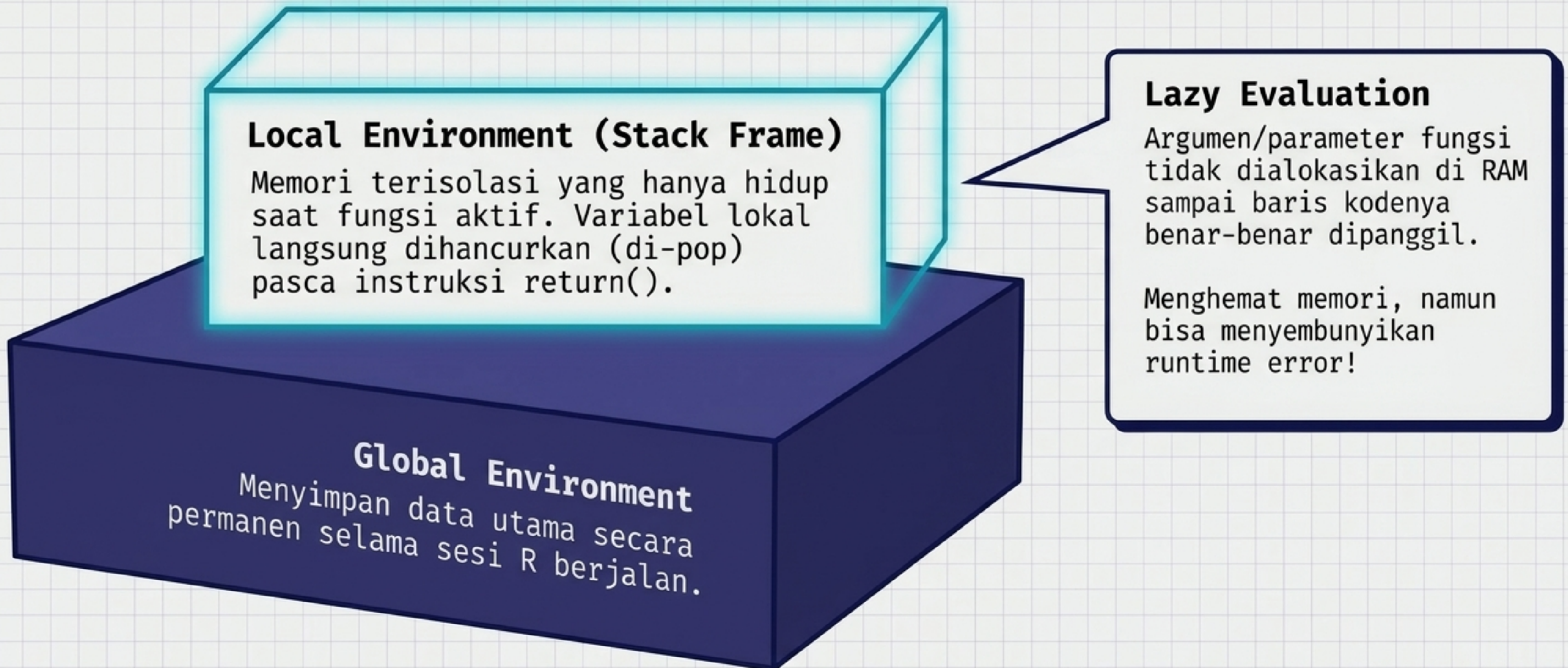
R menggunakan **Column-Major Order**

Pada Nested Loop matriks, selalu iterasi dari **Kolom** lalu **Baris** (Outer j, Inner i).

Iterasi berbasis kolom mencegah **Cache Miss** pada L1/L2 CPU, mempercepat eksekusi program hingga berkali-kali lipat.

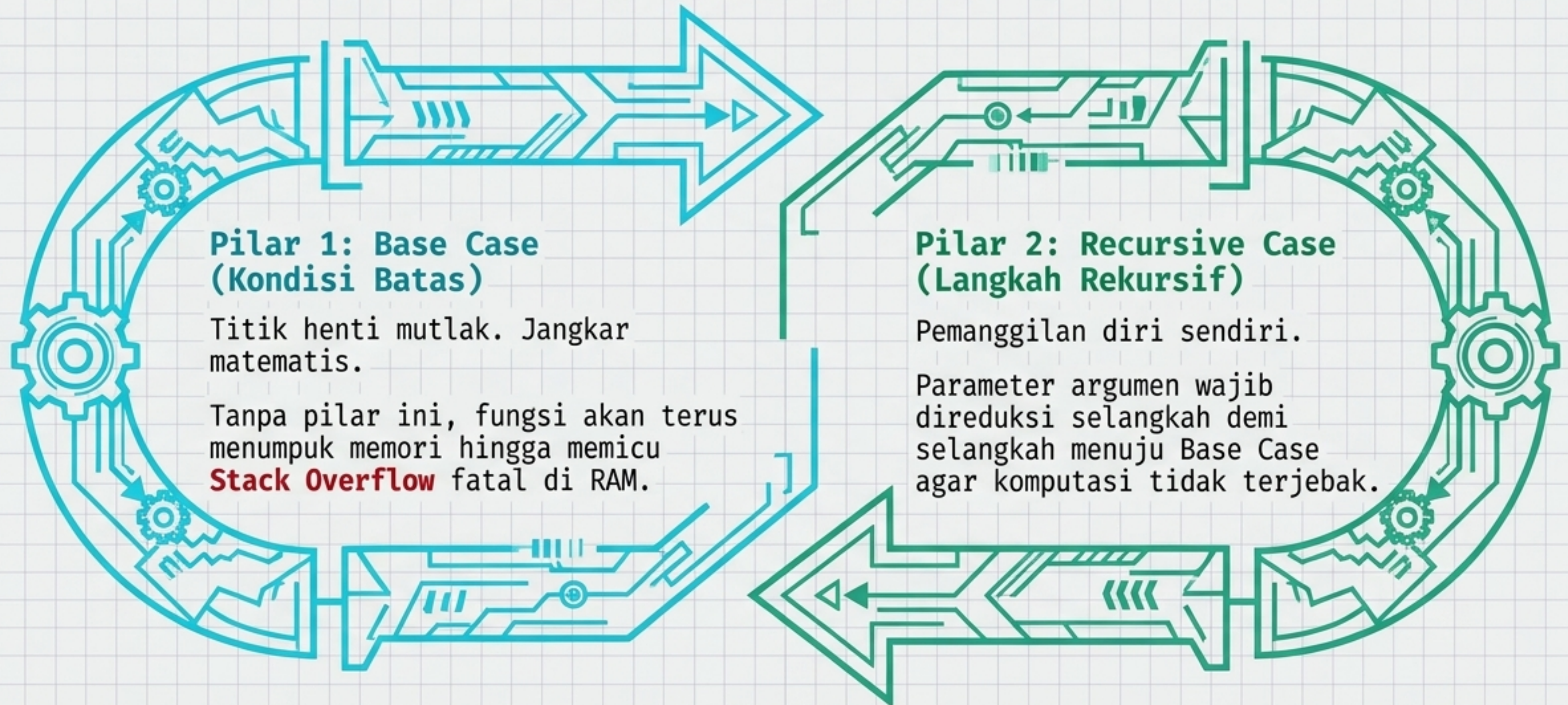
# Pemrograman Modular & Lexical Scoping

Fungsi di R adalah First-Class Citizen (diperlakukan setara dengan objek data).



# Anatomi Rekursif: Memanggil Diri Sendiri

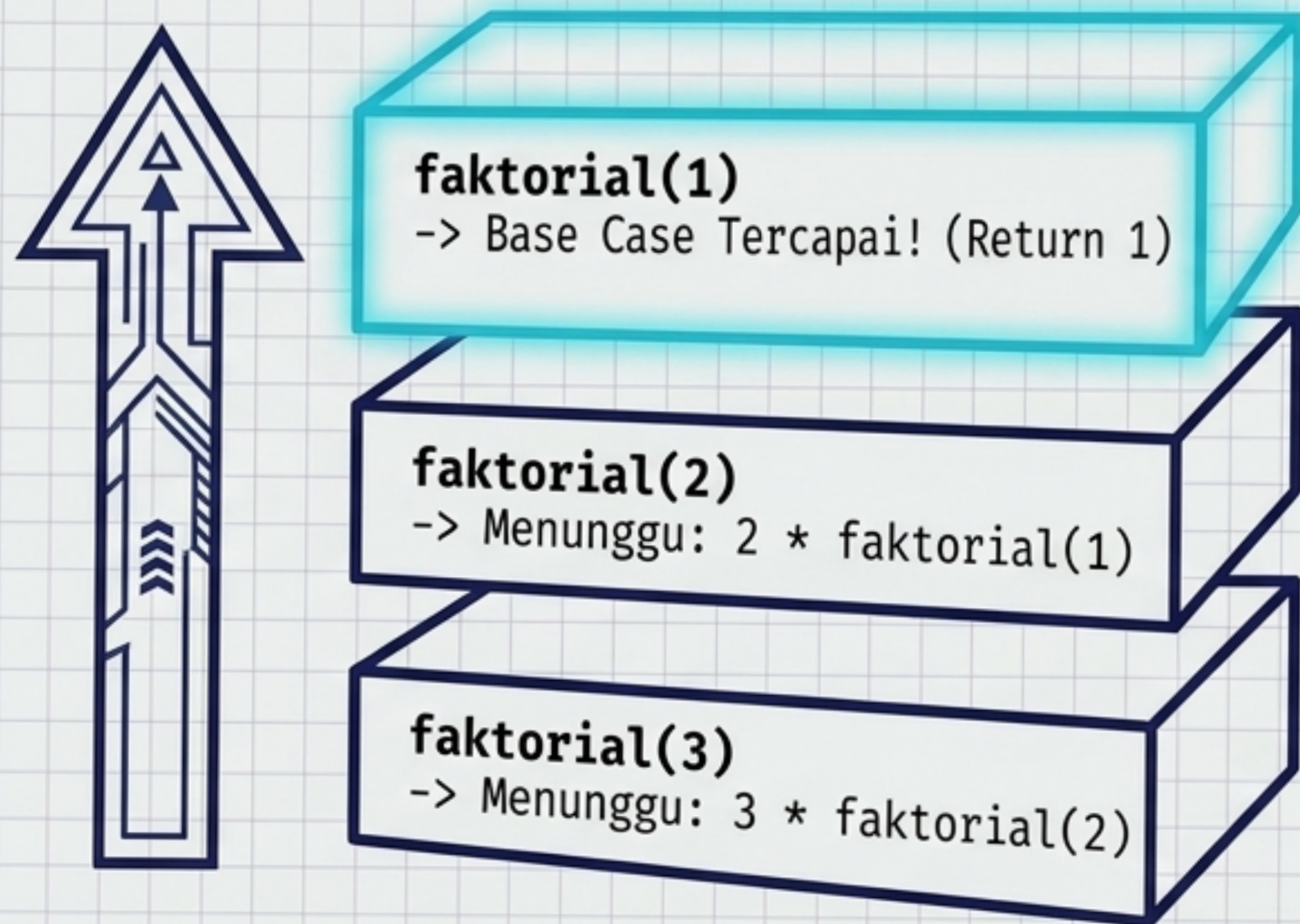
Sub-program memecah masalah besar menjadi sub-masalah identik melalui strategi Divide, Conquer, Combine.



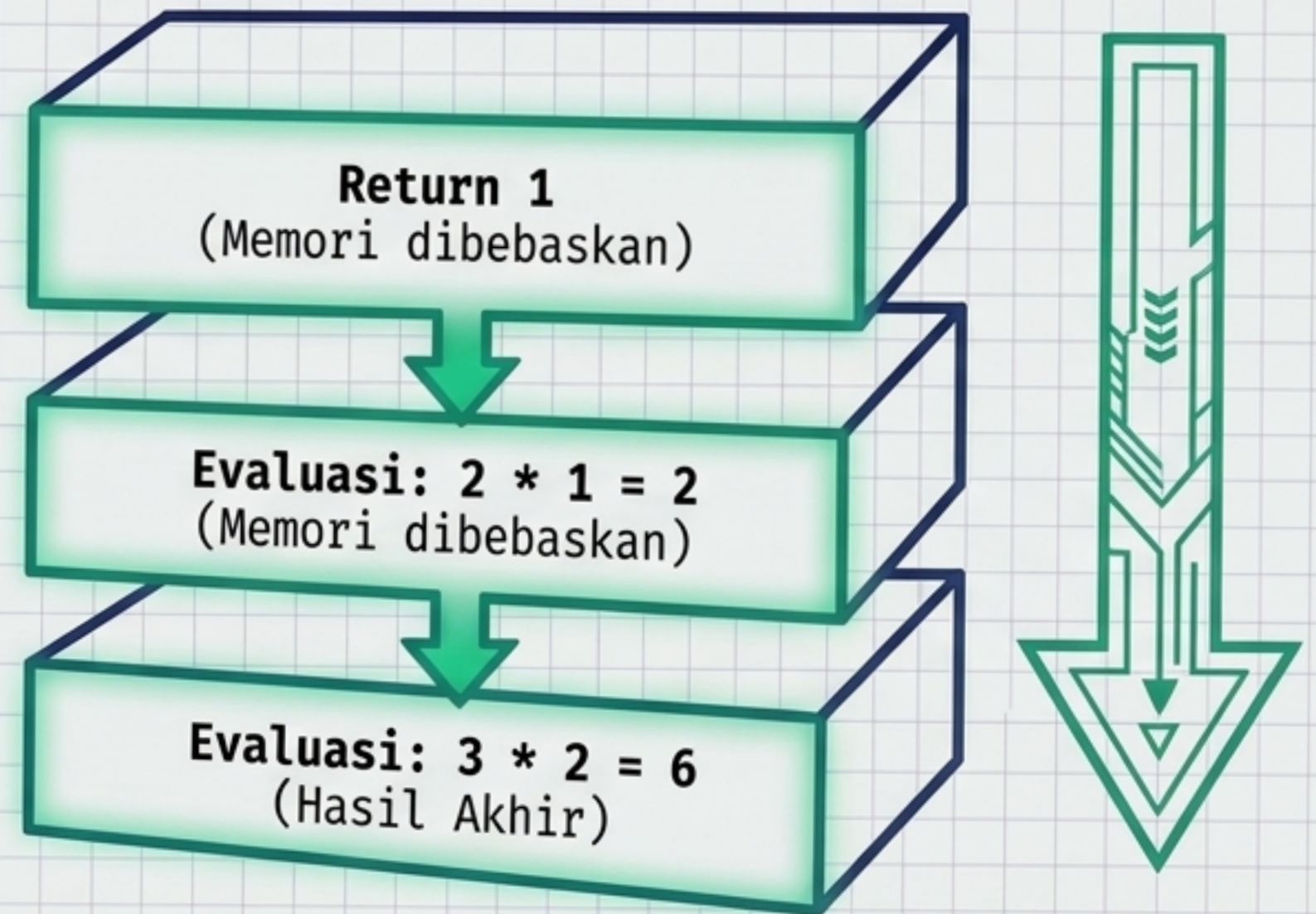
# X-Ray Memori: Fase Winding & Unwinding (Call Stack)

Mekanisme RAM saat mengeksekusi faktorial(3)

## Fase Winding (Penumpukan / PUSH)



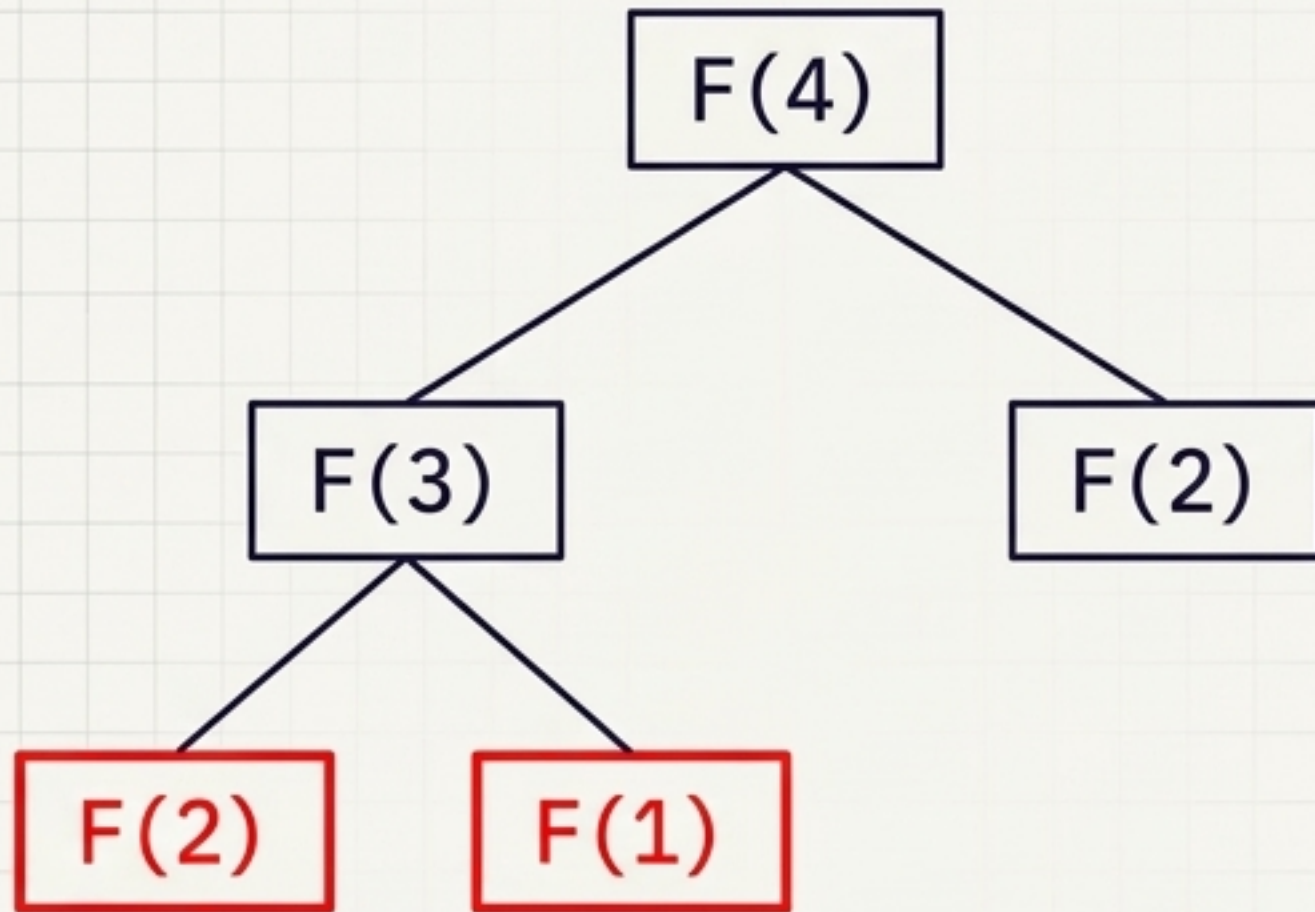
## Fase Unwinding (Penguraian / POP)



Memori fisik (Stack Frames) ditumpuk ke atas, lalu dibebaskan satu per satu ke bawah.

# Rekursif Ganda & Optimasi Memoization

Masalah: Redundansi  $O(2^N)$



Rekursif ganda memicu pertumbuhan eksponensial.

CPU menghitung sub-masalah yang sama berulang kali secara redundan.

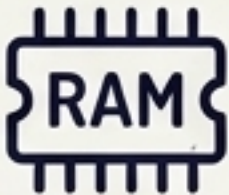
Solusi: Memoization  $O(N)$

| Cache Matrix   |               |        |
|----------------|---------------|--------|
| Slot RAM (Key) | Nilai (Value) | Status |
| F(0)           | 0             | Cached |
| F(1)           | 1             | Cached |
| F(2)           | 1             | Cached |
| F(3)           | 2             | Cached |
| F(4)           | 3             | Cached |

Menyimpan hasil kalkulasi yang sudah selesai ke dalam Lookup Table / Cache di RAM.

Mengubah kompleksitas waktu CPU yang tadinya mencekik ( $O(2^N)$ ) menjadi linier dan instan ( $O(N)$ ).

# Matriks Komparasi Ultima: Iterasi vs Rekursi

| Dimensi Evaluasi                                                                    |                 | Iterasi (Looping)                 | Rekursi                                     |
|-------------------------------------------------------------------------------------|-----------------|-----------------------------------|---------------------------------------------|
|    | Mekanika        | Variabel konter & kondisi         | Panggilan diri & reduksi parameter          |
|    | Memori (Space)  | $O(1)$ - Sangat Hemat, RAM stabil | $O(N)$ - Rakus Call Stack, rentan Overflow  |
|  | Performa (Time) | Cepat (tanpa overhead instruksi)  | Terbebani overhead inisialisasi Stack Frame |
|  | Readability     | Rentan rumit akibat nested loops  | Elegan, ringkas, matematis                  |

Gunakan iterasi untuk pemrosesan data volume besar; gunakan rekursi untuk penelusuran struktur data kompleks dan algoritma matematis.

# Studi Kasus Praktis (Problem Briefing)

## KASUS 1: Optimalisasi Looping Spasial

**Peran:** Data Scientist BMKG.



**Misi:** Diberikan Matriks Curah Hujan spasial berukuran 4x3.

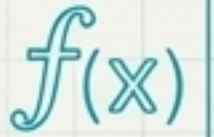
**Instruksi:** Transformasikan nilai  $> 20$  menjadi 'Basah' dan  $\leq 20$  menjadi 'Kering'.

**Syarat Arsitektur:** Wajib menggunakan traversal yang paling efisien di RAM komputer.



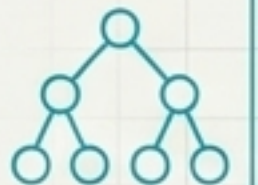
## KASUS 2: Formulasi Rekursif Euklides

**Peran:** Analis Algoritma Matematis.



**Misi:** Rancang sebuah fungsi Pembagi Bersama Terbesar (GCD) dari bilangan a dan b menggunakan Algoritma Euklides secara rekursif.

**Syarat Arsitektur:** Wajib memiliki Base Case yang presisi dan Guard Clause untuk mencegah crash.



# Solusi & Pelacakan Memori (Trace Table)

## Cuplikan Kode R

```
// Solusi 1: Nested Loop Matriks (Column-Major)
for(j in 1:ncol) {      # Outer: Kolom
  for(i in 1:nrow) {    # Inner: Baris
    # Pre-alokasi status_matrix dimutasi di sini
  }
}

// Solusi 2: GCD Rekursif
hitung_gcd <- function(a, b) {
  if(b == 0) return(a)  # Base Case
  return(hitung_gcd(b, a %% b)) # Rekursif
}
```

## Trace Table GCD(48, 18)

|                        |                                          |
|------------------------|------------------------------------------|
| Level 1:<br>a=48, b=18 | --> Panggil<br>gcd(18, 12)               |
| Level 2:<br>a=18, b=12 | --> Panggil<br>gcd(12, 6)                |
| Level 3:<br>a=12, b=6  | --> Panggil<br>gcd(6, 0)                 |
| Level 4:<br>a=6, b=0   | --> Base Case<br>Tercapai!<br>(Return 6) |

# Diagnostic Check: Mini Kuis Analisis Bug

1. Apa dampak komputasional menggunakan `v <- c(v, x)` di dalam loop untuk 100.000 data? \_\_\_\_\_



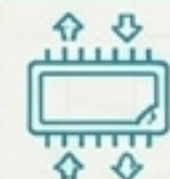
2. Apa logical bug yang terjadi dari `for(i in 1:length(x))` jika `x` adalah vektor kosong? \_\_\_\_\_



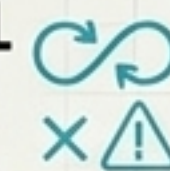
3. Jika kondisi `while` bernilai `FALSE` di iterasi pertama, berapa kali badan loop dieksekusi? \_\_\_\_\_



4. Agar Cache Hit maksimal pada RAM, nested loop di R harus melakukan iterasi dari baris atau kolom terlebih dahulu? \_\_\_\_\_



5. Apa nama error fatal yang terjadi jika fungsi rekursif tidak memiliki Base Case? \_\_\_\_\_



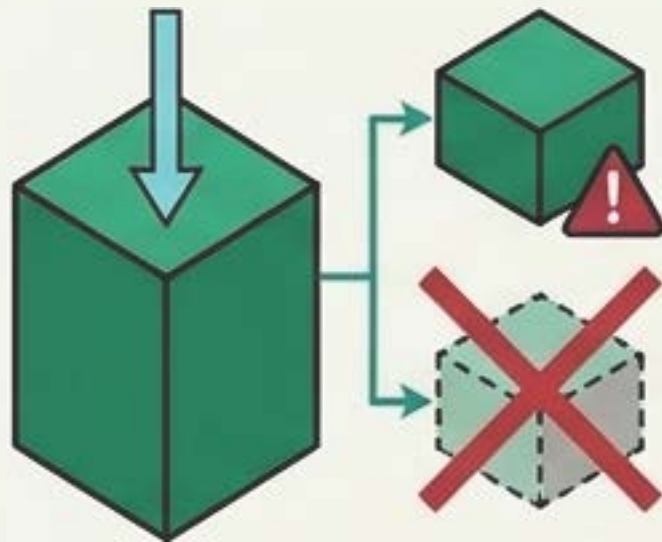
# Jawaban & Rasionalisasi Sistem

1. Apa dampak komputasional menggunakan `v <- c(v, x)` di dalam loop  
**Kompleksitas  $O(N^2)$ .** → R akan terus mencari blok memori baru dan menyalin ulang seluruh data (Memory copy overhead).
2. Apa logical bug yang terjadi dari `for(i in 1:length(x))` jika x  
**Dieksekusi tepat 2 kali.** → Dievaluasi menjadi deret menurun 1:0. Selalu gunakan `seq_along(x)` untuk keamanan.
3. Jika kondisi while bernilai FALSE di iterasi pertama, berapa kali  
**Tepat 0 kali.** → Karena while adalah Pre-test loop yang menguji syarat di gerbang masuk.
4. Agar Cache Hit maksimal pada RAM, nested loop di R harus melakukan  
**Kolom (Column-Major Order).** → Memori R dialokasikan kolom-demi-kolom. Ini mencegah Cache Miss pada prosesor.
5. Apa nama error fatal yang terjadi jika fungsi rekursif tidak memiliki  
**Stack Overflow.** → Call Stack RAM akan terisi penuh oleh Stack Frames yang terus menumpuk dan tidak pernah di-pop.

# Kesimpulan: 4 Aturan Arsitektur Repetisi

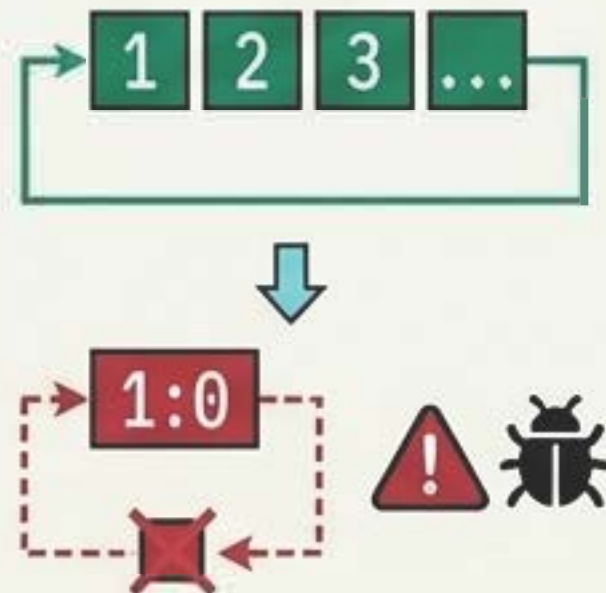
## 1. PRE-ALLOCATION

Selalu siapkan alokasi memori penuh di awal sebelum melakukan Looping data raksasa di R untuk menghindari overhead penyalinan RAM.



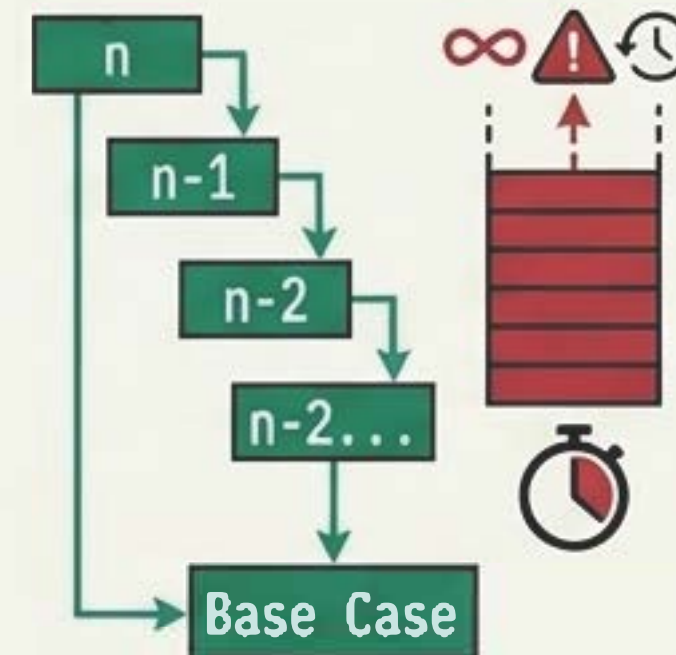
## 2. GUNAKAN SEQ\_ALONG()

Tinggalkan kebiasaan `1:length()`. Gunakan `seq_along()` untuk iterasi indeks yang kebal terhadap bahaya vektor kosong (zero-length bug).



## 3. JANGKAR REKURSIF

Fungsi rekursif tanpa Base Case adalah bom waktu memori. Selalu pastikan parameter direduksi selangkah demi selangkah menuju batas akhir.



## 4. VEKTORISASI PERTAMA

R lambat pada nested loops. Jika ada fungsi bawaan tersertifikasi (seperti `ifelse`, `rowSums`, `dist`), prioritaskan Vektorisasi sebelum merancang loop manual.

